

中国科学院大学

《计算机体系结构(研讨课)》实验报告

姓名 袁晨圃;王永生;李知谦 箱子号 19 专业 计算机科学与技术
实验项目编号 10, 11 实验名称 在流水线中添加普通用户态指令

一、实验目标与总体设计

本次报告包含实验 10 与实验 11。在实验 9 已有五级流水线、前递与基本算术访存指令实现的基础上,实验 10 补充算术逻辑类与乘除法类指令,包含 slti, sltui, andi, ori, xori, sll/srl/sra 以及 pcaddu12i, 同时新增乘法与除法相关指令 mul.w, mulh.w, mulh.wu, div.w, mod.w, div.wu, mod.wu; 实验 11 继续扩展分支转移指令 blt, bge, bltu, bgeu, 并实现字节与半字的访存指令 ld.b, ld.h, ld.bu, ld.hu, st.b, st.h。目标是在不破坏既有流水与前递体系的前提下完成译码、执行、访存与写回通路的增量改造,并在仿真与上板环境中通过功能验证。

总体思路如下。译码阶段在不改变原有操作码判定方式的前提下,增补指令判定与立即数类别;在 EX 阶段扩展 ALU 操作位宽,引入乘法组合逻辑以及除法 Vivado IP 的握手与结果回收状态机,同时与流水的 ready/-valid 机制对齐;在 MEM 阶段实现按地址对齐的字节与半字选通、以及带符号与无符号扩展;在 ID 阶段实现新的比较分支与 PC 目标计算;在冲突处理方面保证多周期除法对前递与阻塞信号的正确驱动。

二、 逻辑电路结构与仿真波形的截图及说明

1 处理器的结构设计框图

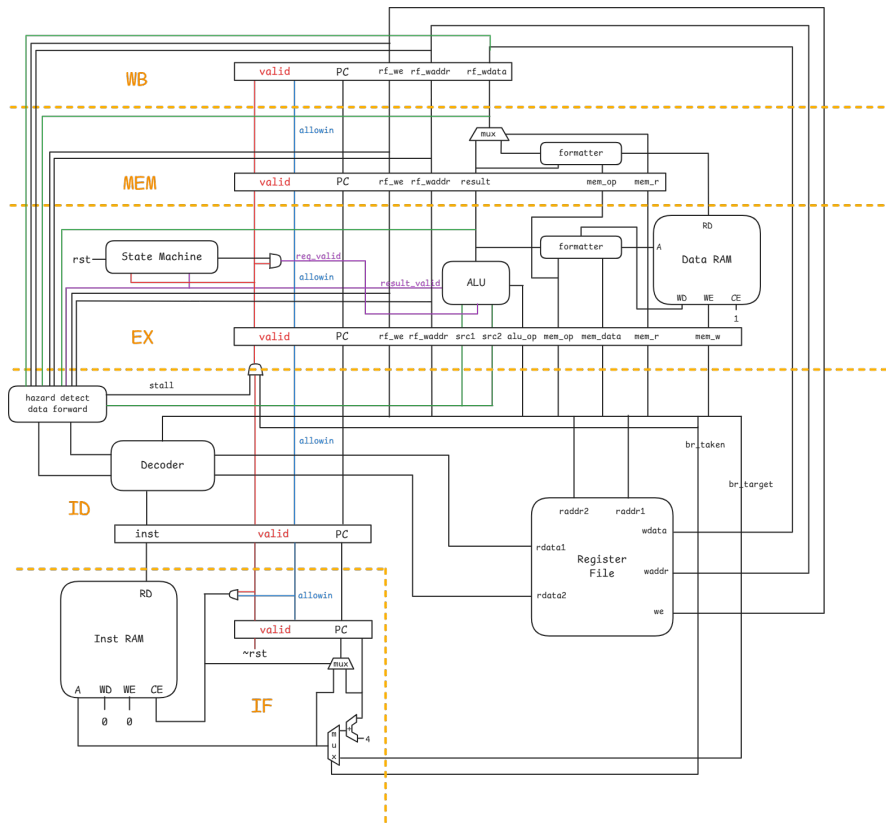


图 1: Prj3 处理器结构

2 指令支持与实现细节(实验 10)

算术逻辑类与移位类扩展在 ID 译码中完成操作码识别,新增 ui12 类立即数以支持 andi,ori,xori,并将寄存器移位 sll/srl/sra 与立即数移位 slli/srli/srai 汇入同一 ALUOp。以下为译码与 ALUOp 映射的相关代码:

```
// stage_id.v: 新增译码与 alu_op 映射
assign inst_slti = op_31_26_d[6'h00] & op_25_22_d[4'b1000];
assign inst_sltui = op_31_26_d[6'h00] & op_25_22_d[4'b1001];
assign inst_andi = op_31_26_d[6'h00] & op_25_22_d[4'b1101];
assign inst_ori = op_31_26_d[6'h00] & op_25_22_d[4'b1110];
assign inst_xori = op_31_26_d[6'h00] & op_25_22_d[4'b1111];
assign inst_sll_w = op_31_26_d[6'h00] & op_25_22_d[4'h0] & op_21_20_d[2'h1] & op_19_15_d[5'b01110];
assign inst_srl_w = op_31_26_d[6'h00] & op_25_22_d[4'h0] & op_21_20_d[2'h1] & op_19_15_d[5'b01111];
assign inst_sra_w = op_31_26_d[6'h00] & op_25_22_d[4'h0] & op_21_20_d[2'h1] & op_19_15_d[5'b10000];

assign need_ui12 = inst_andi | inst_ori | inst_xori;
assign need_si12 = inst_addi_w | inst_ld_w | inst_st_w | inst_slti | inst_sltui;
assign need_si20 = inst_lui12i_w | inst_pcaddu12i;

assign imm = src2_is_4 ? 32'h4
    : need_si20 ? {i20[19:0], 12'b0}
    : need_ui12 ? {20'b0, i12[11:0]}
    : {{20{i12[11]}}, i12[11:0]};

assign alu_op[ 2] = inst_slt | inst_slti;
assign alu_op[ 3] = inst_sltu | inst_sltui;
assign alu_op[ 4] = inst_and | inst_andi;
assign alu_op[ 6] = inst_or | inst_ori;
assign alu_op[ 7] = inst_xor | inst_xori;
assign alu_op[ 8] = inst_slli_w | inst_sll_w;
assign alu_op[ 9] = inst_srli_w | inst_srl_w;
assign alu_op[10] = inst_srai_w | inst_sra_w;
```

pcaddu12i 以 PC 为 src1、以上移 12 位的 20 位立即数为 src2 参与加法,从而统一走 ALU 加法通路并自然适配前递与旁路:

```
// stage_id.v: pcaddu12i 作为加法输入
assign inst_pcaddu12i = op_31_26_d[6'b000111] & ~inst[25];
assign src1_is_pc = inst_jirl | inst_bl | inst_pcaddu12i;
assign alu_op[0] = inst_add_w | inst_addi_w | inst_ld_w | inst_st_w
    | inst_jirl | inst_bl | inst_pcaddu12i;
```

ALU 扩展为支持乘法的组合逻辑以及除法结果复用。乘法路径直接用 33 位有符号扩展实现 mul.w、mulh.w、mulh.wu 的低 32 位与高 32 位;除法路径使用 Vivado IP (udiv/sdiv),通过 valid/ready 握手驱动,ALU 内部维护请求、接收状态机并输出结果有效信号。

```
// alu.v: 乘法与除法整合, 以及结果多路选择
assign {_discard, mulh_result, mul_result} =
    $signed({~op_mulhu & alu_src1[31], alu_src1}) *
    $signed({~op_mulhu & alu_src2[31], alu_src2});
```

```

sdiv u_sdiv(...);
udiv u_udiv(...);

assign alu_result = ({32{op_add|op_sub }} & add_sub_result)
    | ({32{op_mul      }} & mul_result)
    | ({32{op_mulh|op_mulhu}} & mulh_result)
    | ({32{op_div      }} & sdiv_quotient)
    | ({32{op_divu     }} & udiv_quotient)
    | ({32{op_mod      }} & sdiv_remainder)
    | ({32{op_modu     }} & udiv_remainder)
    | ... ;

always @(*) begin
    case (current_state)
        STATE_INIT: begin
            next_state = STATE_SIMPLE;
        end
        STATE_SIMPLE: begin
            next_state = (alu_req_valid && div_enable) ? STATE_DIV_REQ : STATE_SIMPLE;
        end
        STATE_DIV_REQ: begin
            if (sdiv_enable && (sdiv_divisor_valid & sdiv_divisor_ready & sdiv_dividend_ready &
                sdiv_dividend_valid)) next_state = STATE_DIV_RECV;
            else if (udiv_enable && (udiv_divisor_valid & udiv_divisor_ready & udiv_dividend_ready &
                udiv_dividend_valid)) next_state = STATE_DIV_RECV;
            else next_state = STATE_DIV_REQ;
        end
        STATE_DIV_RECV: begin
            if (div_result_valid) next_state = STATE_SIMPLE;
            else next_state = STATE_DIV_RECV;
        end
        default: begin
            next_state = STATE_INIT;
        end
    endcase
end

assign sdiv_divisor_valid = (current_state == STATE_DIV_REQ) && sdiv_enable;
assign udiv_divisor_valid = (current_state == STATE_DIV_REQ) && udiv_enable;
assign sdiv_dividend_valid = (current_state == STATE_DIV_REQ) && sdiv_enable;
assign udiv_dividend_valid = (current_state == STATE_DIV_REQ) && udiv_enable;

assign alu_result_valid = ((current_state == STATE_SIMPLE) & ~div_enable)
    | ((current_state == STATE_DIV_RECV) & div_result_valid);

```

由于除法是多周期,EX 级同样引入状态机,以及 readygo 信号,防止错误前递或提前放行。

```

// stage_ex.v: 与 ALU 的请求与结果握手; 准备就绪才放行
assign readygo = ((current_state == STATE_REQ) & alu_output_valid)
    | (current_state == STATE_DONE);

```

```
assign forward_ready = readygo & ~output_mem_read; // 未完成或读存储都不前递
```

3 分支与字节半字访存(实验 11)

分支类指令 blt、bge、bltu、bgeu 的本质是对寄存器对进行有符号或无符号比较后,按 16 位偏移更新 PC。为降低译码关键路径,我们将比较逻辑抽象为一个独立比较器模块,实现等于、小于(有符号与无符号)三类比较输出,在 ID 阶段进行组合判定并给出分支目标。

```
// tools.v: 比较器与 ID 阶段分支判定
module comparator(
    input [31:0] a, input [31:0] b,
    output a_eq_b, output a_lt_b_signed, output a_lt_b_unsigned);
    assign a_eq_b = (a == b);
    wire a_sign = a[31], b_sign = b[31];
    assign a_lt_b_signed = (a_sign & ~b_sign) | ((a_sign == b_sign) & (a < b));
    assign a_lt_b_unsigned = (a < b);
endmodule

assign br_taken = (inst_beq && rj_eq_rd)
    || (inst_bne && !rj_eq_rd)
    || (inst_blt && rj_lt_rd)
    || (inst_bge && !rj_lt_rd)
    || (inst_bltu && rj_ult_rd)
    || (inst_bgeu && !rj_ult_rd)
    || inst_jirl || inst_bl || inst_b;
assign br_target = (inst_beq || inst_bne || inst_blt || inst_bge
    || inst_bltu || inst_bgeu || inst_bl || inst_b)
    ? (pc + br_offs) : (rj_value + jirl_offs);
```

访存扩展方面, EX 阶段依据低地址位计算写选通信号与写数据排列,以避免在 MEM 阶段再做复杂门控; MEM 阶段对读取数据按字节与半字进行选择与符号/零扩展,最终统一回写。

```
// stage_ex.v: 按地址生成写使能与写数据 (st.b/st.h/st.w)
assign data_sram_we = op_st_w ? 4'b1111
    : op_st_h ? (alu_result[1] ? 4'b1100 : 4'b0011)
    : op_st_b ? (alu_result[1:0] == 2'b00 ? 4'b0001
        : alu_result[1:0] == 2'b01 ? 4'b0010
        : alu_result[1:0] == 2'b10 ? 4'b0100
        : alu_result[1:0] == 2'b11 ? 4'b1000
        : 4'b1000)
    : 4'b0000;
assign data_sram_wdata = op_st_b ? {4{mem_data[7:0]}}
    : op_st_h ? {2{mem_data[15:0]}}
    : op_st_w ? mem_data
    : 32'h0;

// stage_mem.v: 字节/半字选择与扩展 (ld.b/ld.h/ld.bu/ld.hu/ld.w)
wire [7:0] selected_byte = (alu_result[1:0] == 2'b00) ? mem_rdata[7:0]
    : (alu_result[1:0] == 2'b01) ? mem_rdata[15:8]
```


乘法高位与有符号性问题。mulh.w 与 mulh.wu 的差异在于是否以有符号数相乘并取高 32 位。采用 33 位扩展并以 op_mulhu 控制被乘数符号位注入可以在同一乘法器上完成三类乘法,避免额外硬件重复。

除法多周期与流水握手问题。将除法 IP 的 ready/valid 握手外露到 ALU,并在 EX 级用状态机与流水 ready/valid 相连,只有在结果真正可用时才拉高 readygo,从而保证前递与放行时机一致。

字节与半字的端序与对齐问题。写入时根据地址低位生成按字节的写使能与数据复制布局;读取时先按地址抽取目标字节或半字,再依据 ld.b/ld.bu 与 ld.h/ld.hu 决定是否注入符号位。这样可以保证对齐与扩展逻辑的独立与清晰。

保持在 ID stall 期间不产生 br_taken,避免错误跳转。

此外,我们在做实验 11 时还遇到时序违例的问题,经过长时间调试排查后,发现原来是环境配置不正确。这也提醒我们,当遇到 bug 时,不妨先检查环境配置等外部因素,而不是一来就怀疑自己代码出错,这样或许能节省不少调试时间。

四、 实验所耗时间与组内分工

在课后,花费了大约__15__小时完成此次实验。

袁晨圃:完成实验 10

王永生:完成实验 11

李知谦:撰写报告